



Build a Microservices CRUD App

React · ASP.NET Core · Ocelot Gateway · SQL Server · Docker · Docker Hub · CI/CD ·
Render

A hands-on lab · use  /  or Space to navigate

GOAL

What we're building

- Users **register / log in** → get a **JWT** (Authentication service)
- Logged-in users **Create / Read / Update / Delete** products (CRUD service)
- The Product API is **protected** — no valid token, no access
- An **Ocelot API Gateway** (:7000) is the single entry point for the browser

Frontend

React (Vite) — calls only the gateway

3 backends

Gateway + 2 ASP.NET Core Web APIs

Database

SQL Server (one DB per service)

Ops

Docker · GitHub Actions · Docker Hub · Render

CONCEPT

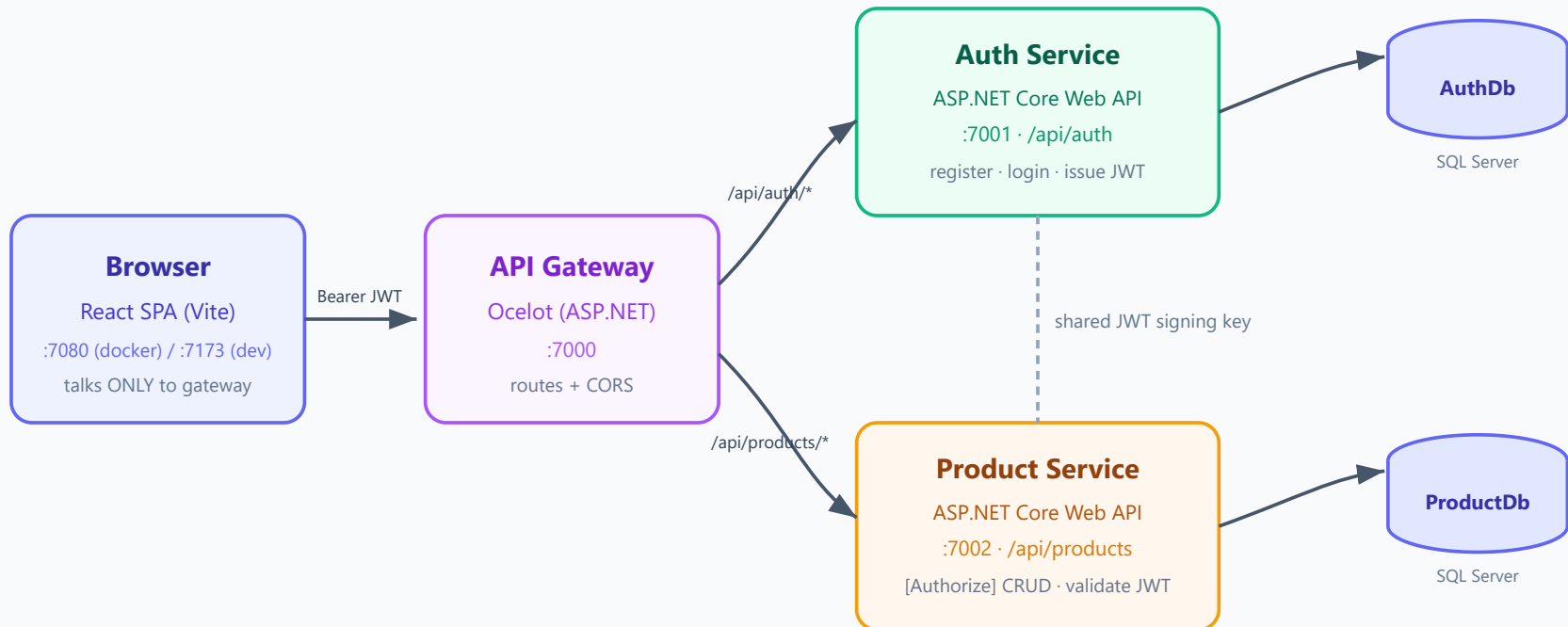
What is a microservice?

- A **small, independently deployable** service that owns **one** job
- **Database-per-service**: each service owns its own data — no shared tables
- Services talk over **HTTP/REST**
- An **API gateway** gives clients a single entry point and routes to the right service

Here: an **Auth** service and a **Product** service (each with its own SQL Server database) behind an **Ocelot** gateway.

How it fits together

System Architecture — Microservices CRUD App with API Gateway



The browser only ever calls the gateway (:7000). Ocelot routes to each service; each service owns its own database.

Auth signs JWTs; Product validates them statelessly with the same key.

THE KEY IDEA

JWT across services

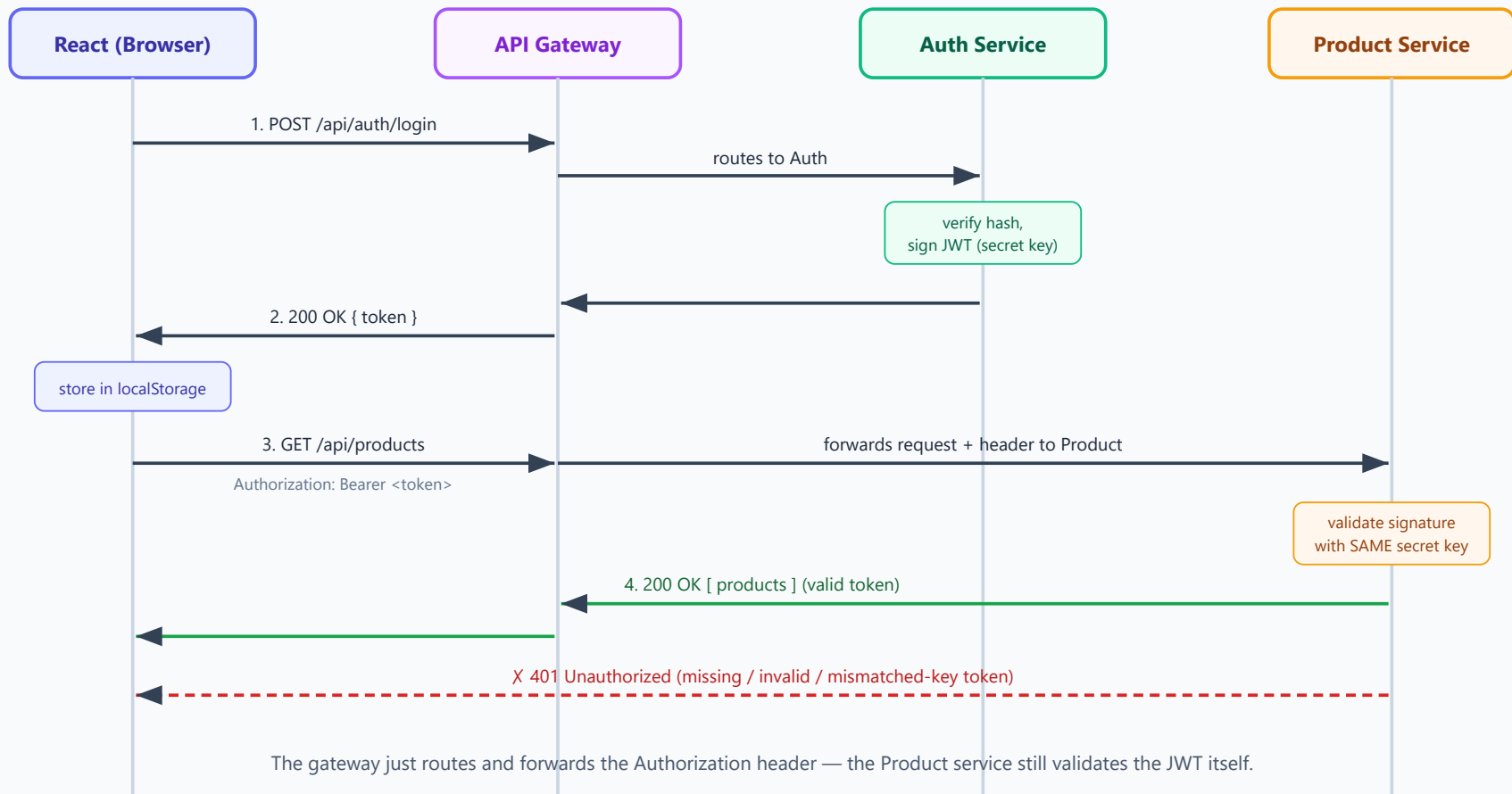
1. Auth service **signs** a token with a secret key on login
2. React stores it and sends `Authorization: Bearer <token>`
3. Product service **validates** it with the **same key** — no call back to Auth

 **Golden rule:** `Jwt__Key`, `Jwt__Issuer`, `Jwt__Audience` must be **identical** in both services — or you get **401**.

FLOW

Login → token → protected call

JWT Authentication Flow (through the API Gateway)



PART A

SQL Server in Docker

No manual install — run it as a container:

```
docker run -e "ACCEPT_EULA=Y" \  
  -e "MSSQL_SA_PASSWORD=Your_password123" \  
  -p 14330:1433 --name lab-sqlserver -d \  
  mcr.microsoft.com/mssql/server:2022-latest
```

Host port **14330** (non-default → no clashes). Each service creates its own DB on first run via

`EnsureCreated()`.

PART B

Auth service (ASP.NET)

- `POST /api/auth/register` and `POST /api/auth/login`
- Passwords hashed with **PBKDF2** + random salt — never stored raw
- `TokenService` signs the JWT; returns `{ token, expiresAt }`
- EF Core + SQL Server → `AuthDb`

PART C

Product CRUD service

- Full CRUD on `/api/products`
- `[Authorize]` on the controller → every route needs a valid JWT
- `AddJwtBearer(...)` validates issuer, audience, lifetime, signature
- Own database → `ProductDb` (seeded with 2 sample products)

Demo: `GET /api/products` → **401** → Authorize → **200** 🎉

PART D

API Gateway (Ocelot) — :7000

- Single entry point: the browser calls **only** `localhost:7000`
- A JSON routing table maps `/api/auth/*` → :7001 and `/api/products/*` → :7002
- Forwards the `Authorization` header — but the Product service **still validates** the JWT
- CORS lives here now (one place, not per-service)

Three routing files, picked by `ASPNETCORE_ENVIRONMENT`:

`ocelot.json` (local) · `ocelot.Docker.json` (compose) · `ocelot.Production.json` (Render)

React frontend — :7173

api.js

fetch wrapper; ONE base URL (the gateway);
attaches the Bearer token

AuthContext

stores user + token in localStorage

Pages

Login · Register · Products (CRUD)

ProtectedRoute

redirects anonymous users to /login

⚠ Vite reads `VITE_API_URL` at **build time** — it's baked into the bundle.

The app



Microservice Lab

Hi, student

Logout

Add product

Products

ID	Name	Description	Price	Stock		
1	Wireless Mouse	2.4GHz ergonomic mouse	\$19.99	50	Edit	Delete
2	Mechanical Keyboard	RGB, blue switches	\$59.90	30	Edit	Delete

Figure: Products CRUD screen — protected by JWT. Click Edit to load a row into the form; Delete removes it.

Docker Compose — one command

```
docker compose up --build
```

- Frontend → `localhost:7080` · Gateway → `localhost:7000`
- Auth → `localhost:7001` · Product → `localhost:7002` · SQL → `localhost,14330`
- Gateway runs with `ASPNETCORE_ENVIRONMENT=Docker` → loads `ocelot.Docker.json`

Inside the compose network the DB host is `sqlserver,1433`, **not** `localhost`. #1 gotcha!

DOCKER

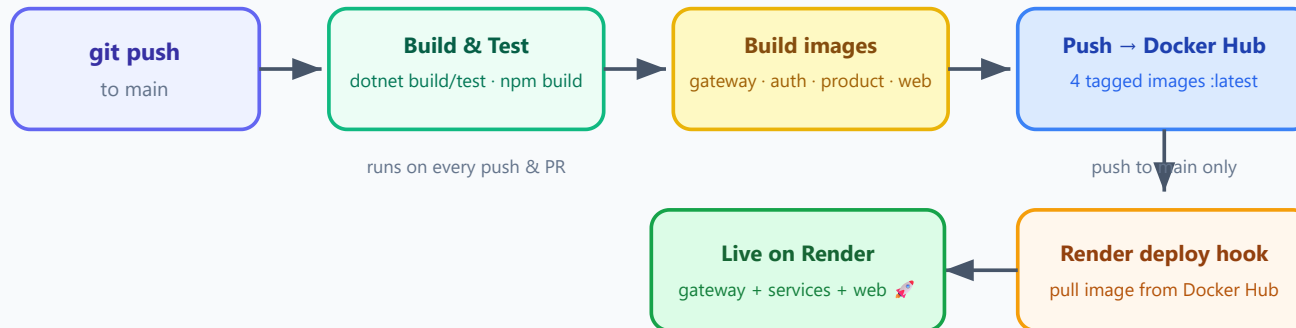
Multi-stage builds

- **Build stage:** heavy SDK / Node — compiles the app
- **Runtime stage:** tiny image — runs only the output
- Result: smaller, faster, more secure images

Frontend: Node builds → **nginx** serves the static files.

CI/CD — GitHub Actions → Docker Hub

CI/CD Pipeline — GitHub Actions → Docker Hub → Render



Defined in `.github/workflows/ci-cd.yml` — jobs: `build-backend · test-backend · build-frontend · docker-push · deploy`.

Build & test → build **4 images** → **push to Docker Hub** (`:latest` + commit SHA) → tell Render to pull.
Build once, deploy the exact image you tested.

PART I

Deploy to Render — from Docker Hub

- All 4 services deployed as **existing images** pulled from Docker Hub
- Order: services → gateway (needs service URLs) → frontend (gateway URL baked in)
- Config via **environment variables**: conn string, JWT key, gateway `Routes__*`, CORS
- SQL Server hosted externally (Azure SQL free) — Render gets the connection string

Deploy hooks + GitHub secrets → every push builds, pushes & redeploys. 🚀

DON'T GET STUCK

Top 4 pitfalls

1. **401 everywhere:** JWT key/issuer/audience differ between Auth & Product
2. **CORS error:** frontend URL not in the **gateway's** `AllowedOrigins`
3. **Gateway 404/502:** wrong `ocelot.*.json` loaded, or wrong downstream host
4. **DB unreachable in Docker:** used `localhost` instead of `sqlserver`



You built a real microservices app

Ocelot gateway · 2 ASP.NET services · JWT auth · SQL Server · React · Docker · Docker
Hub CI/CD · Render

Questions? Open [README.md](#) for the full step-by-step.